visualize_jct_acceleration_cdf

February 12, 2026

1 Figure 13. (b) Impact on individual jobs relative to the non-elastic scheduler

1.1 1. Setup and Load Data

Choose parameters for the analysis and load the job statistics data from checkpoint files.

[1]:
```
# choose step size for the plot, only for visualization, not for calculation
step_size = 1000

# whether to show figure
show_fig = True

# set the total number of GPUs in cluster

from Suika import *
config = "../configs/cluster64.json"
total_n_acc = parse_cluster_config(config)[0].get_total_acc_num()
print(f"Total number of GPUs in cluster: {total_n_acc}")

import torch
import os

job_stats = {}

testsets = ["suika", "sia", "rubick", "fcfs"]

policy_map = {
    "suika": "Suika",
    "sia": "Sia",
    "rubick": "Rubick",
    "dp": "Suika-DP",
    "metis": "Suika-Metis",
    "fcfs": "FIFO",
}

policy_styles = {
    "suika":  {"color": "#34495e", "hatch": "", "linestyle": "-"},
```

```
    "sia":    {"color": "#e67e22", "hatch": "/", "linestyle": "--"},
    "rubick": {"color": "#f1c40f", "hatch": "x", "linestyle": "-."},
    "fcfs":   {"color": "#e74c3c", "hatch": "-", "linestyle": ":"},
}

cluster = "clusterA"
for testcase in testsets:
  file = f"../results/{cluster}/{testcase}-{cluster}.pth"

  try:
    import pickle
    with open(file, "rb") as f:
      job_stats[testcase] = torch.load(f, weights_only=False)
  except FileNotFoundError:
    print(f"{file} not found: please ensure the it exists.")
    # exit(1) # exit is not friendly in notebook
  else:
    print(f"loaded job statistics for {testcase} with␣
 ↪{len(job_stats[testcase])} entries")
```

```
/home/fake/.local/lib/python3.10/site-packages/tqdm/auto.py:21: TqdmWarning:
IProgress not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

```
Total number of GPUs in cluster: 64
loaded job statistics for suika with 45 entries
loaded job statistics for sia with 45 entries
loaded job statistics for rubick with 45 entries
loaded job statistics for fcfs with 45 entries
```

2 2 Acceleration CDF

X values: the reduction in JCT for individual job (may be negative if slow down) Y values: CDF of job count

Use last value in jss as baseline for acceleration comparison. (also plot the last jss!)

[2]:
```
def get_job_acceleration(baseline_js, compare_js, job_filter_list):
  """Calculate JCT reduction (acceleration) for individual jobs compared to␣
 ↪baseline"""
  baseline_stats = analyze_job_statistics(baseline_js)["per_job_statistics"]
  compare_stats = analyze_job_statistics(compare_js)["per_job_statistics"]

  accelerations = []
  missing_jobs = 0

  for job in job_filter_list:
```

```
    if job in baseline_stats and job in compare_stats:
      baseline_jct = baseline_stats[job].get("jct")
      compare_jct = compare_stats[job].get("jct")

      if baseline_jct is not None and compare_jct is not None:
        # Acceleration = reduction in JCT (positive = faster, negative = slower)
        acceleration = baseline_jct - compare_jct
        accelerations.append(acceleration)
      else:
        missing_jobs += 1
    else:
      missing_jobs += 1

  if missing_jobs > 0:
    print(f"Warning: {missing_jobs} jobs missing JCT data for acceleration␣
↪calculation")

  return accelerations

def plot_acceleration_cdf(jss, min_duration=100):
  """Plot acceleration CDF comparing all policies to the last policy as␣
↪baseline"""
  import matplotlib.pyplot as plt
  import numpy as np

  # Use last policy as baseline (includes ALL jobs, no duration filtering)
  baseline_policy = list(jss.keys())[-1]
  baseline_js = jss[baseline_policy]

  # Get all jobs from baseline policy (no filtering for ANY policy)
  baseline_stats = analyze_job_statistics(baseline_js)["per_job_statistics"]
  all_jobs = list(baseline_stats.keys())

  plt.figure(figsize=(8, 4))

  policy_names = list(jss.keys())

  # Plot acceleration CDF for each policy (including baseline)
  for i, policy_name in enumerate(policy_names):
    label = policy_map.get(policy_name, policy_name)
    if policy_name == baseline_policy:
      # Baseline policy: acceleration is always 0 for ALL jobs
      accelerations = [0.0] * len(all_jobs)
      label = f'{label} (baseline)'
    else:
      # Compare other policies to baseline (using ALL jobs, no filtering)
```

```
    accelerations = get_job_acceleration(baseline_js, jss[policy_name],␣
↪all_jobs)

  if not accelerations:
    print(f"Warning: No acceleration data found for policy {policy_name}")
    continue

  # Sort accelerations for CDF calculation
  accel_sorted = np.sort(accelerations)
  n = len(accel_sorted)

  # Calculate CDF values (cumulative probability)
  cdf_values = np.arange(1, n + 1) / n

  # Prepend a point to make the CDF start from 0
  if n > 0:
      # Use a small epsilon or just the first value for a vertical start
      eps = 1
      plot_accel = np.insert(accel_sorted, 0, accel_sorted[0] - eps)
      plot_cdf = np.insert(cdf_values, 0, 0)
  else:
      plot_accel = accel_sorted
      plot_cdf = cdf_values

  style = policy_styles.get(policy_name, {"color": "gray", "linestyle": "-"})
  # Plot CDF line for this policy
  plt.plot(plot_accel, plot_cdf,
           label=label,
           color=style["color"],
           linestyle=style["linestyle"],
           linewidth=3,
           alpha=0.8)

  # Print statistics for this policy
  mean_accel = np.mean(accelerations)
  median_accel = np.median(accelerations)
  percent_improved = np.sum(np.array(accelerations) > 0) / len(accelerations)␣
↪* 100
  percent_degraded = np.sum(np.array(accelerations) < 0) / len(accelerations)␣
↪* 100

  print(f"{label}:")
  print(f"  Mean acceleration: {mean_accel:.2f}s")
  print(f"  Median acceleration: {median_accel:.2f}s")
  print(f"  Jobs improved: {percent_improved:.1f}%")
  print(f"  Jobs degraded: {percent_degraded:.1f}%")
  print()
```

```
    # Customize the plot
    plt.xlabel('JCT Reduction (s)', fontsize=18)
    plt.ylabel('CDF', fontsize=18)
    # plt.title(f'JCT Reduction CDF vs {policy_map.get(baseline_policy,␣
    ↪baseline_policy)}', fontsize=12, fontweight='bold')
    plt.grid(True, alpha=0.3)
    plt.legend(fontsize=15)

    # Use symlog scale for x-axis (linear near zero, log for larger values)
    plt.tick_params(axis='x', labelsize=14)
    plt.tick_params(axis='y', labelsize=14)

    # Use symlog scale for x-axis (linear near zero, log for larger values)
    plt.xscale('symlog', linthresh=100, linscale=0.2)  # Linear threshold
    plt.xlim(-5e3, 1e4)

    # Add vertical line at zero (no acceleration)
    # plt.axvline(x=0, color='black', linestyle='--', alpha=0.5, linewidth=1)

    # Set y-axis to show full probability range
    plt.ylim(0, 1)

    # Add percentage labels on y-axis
    plt.gca().yaxis.set_major_formatter(plt.FuncFormatter(lambda y, _: f'{y*100:.
    ↪0f}%'))

    plt.tight_layout()

    fig_dir = "../fig"
    os.makedirs(fig_dir, exist_ok=True)
    filename = f"{fig_dir}/jct_reduction_cdf.pdf"
    plt.savefig(filename, dpi=600, bbox_inches='tight')
    print(f"Plot saved as {filename}")

    if show_fig:
      plt.show()
```

[3]:
```
# Plot acceleration CDF for all loaded policies vs the last policy as baseline
plot_acceleration_cdf(job_stats, min_duration=100)
```

```
Suika:
  Mean acceleration: 1434.52s
  Median acceleration: 193.49s
  Jobs improved: 71.1%
  Jobs degraded: 28.9%
```

```
Sia:
  Mean acceleration: 826.75s
  Median acceleration: 109.63s
  Jobs improved: 62.2%
  Jobs degraded: 37.8%

Rubick:
  Mean acceleration: 775.16s
  Median acceleration: 7.32s
  Jobs improved: 55.6%
  Jobs degraded: 44.4%

FIFO (baseline):
  Mean acceleration: 0.00s
  Median acceleration: 0.00s
  Jobs improved: 0.0%
  Jobs degraded: 0.0%

Plot saved as ../fig/jct_reduction_cdf.pdf
```

